



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش کار آزمایش پنجم

عنوان:

طراحی ضرب کننده

نام درس:

آزمایشگاه طراحی سامانه های رقمی

تابستان ۱۴۰۵

استاد:

دکتر اجلالی

دستیار استاد:

علی اثنی عشری

گروه ۲

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	مقدمه و روش بوث	۲
۱.۱	نحوه کارکرد روش پایه	۲
۲.۲	روش بوث بهینه شده	۳
۱.۲	متغیرها و منطق پیاده سازی	۳
۲.۲	مثال روش بهینه شده: ضرب عدد ۷ در ۳	۳
۳	پیاده سازی سخت افزاری و بررسی کدها	۵
۱.۳	پیمانه واحد کنترل	۵
۲.۳	پیمانه مسیر داده	۶
۳.۳	پیمانه سطح بالا	۷
۴	آزمایش و میزآزمون	۹
۱.۴	تحلیل موارد آزمون	۱۰
۱.۱.۴	مورد آزمون نخست: ضرب عدد ۷ در ۵	۱۰
۲.۱.۴	مورد آزمون دوم: ضرب عدد ۴- در ۶-	۱۱
۳.۱.۴	مورد آزمون سوم: ضرب عدد ۷ در ۳-	۱۱
۴.۱.۴	مورد آزمون چهارم: ضرب عدد ۵- در ۳	۱۱
۵.۱.۴	مورد آزمون پنجم: ضرب عدد ۰ در ۷-	۱۲
۶.۱.۴	مورد آزمون ششم: ضرب عدد ۸- در ۷	۱۲
۷.۱.۴	مورد آزمون هفتم: ضرب عدد ۵ در ۱-	۱۲
۵	بررسی قابلیت سنتز در نرم افزار Quartus	۱۴

۱ مقدمه و روش بوث

روش^۱ بوث یکی از روش‌های کارآمد برای ضرب اعداد علامت‌دار یا همان اعداد مکمل دو^۲ در سخت‌افزار است. در روش ضرب معمولی، وجود اعداد منفی نیازمند پیش‌پردازش است، اما روش بوث با بررسی همزمان دو بیت متوالی از ضریب، عملیات ضرب را بدون نیاز به تغییر علامت اعداد به درستی انجام می‌دهد.

۱.۱ نحوه کارکرد روش پایه

در این روش ما از چهار ثبات^۳ اصلی استفاده می‌کنیم:

- ثبات M : دربردارنده عدد مضروب^۴.
 - ثبات Q : دربردارنده عدد ضریب^۵.
 - ثبات A : ثبات انباشتگر^۶ که در ابتدا مقدار صفر دارد.
 - بیت Q_{-1} : یک بیت کمکی که در ابتدا صفر است.
- در هر مرحله، دو بیت Q_0 (بیت دارای کمترین ارزش ضریب) و Q_{-1} بررسی می‌شوند:
- اگر مقادیر برابر با ۰۰ یا ۱۱ باشند: ترکیب ۹ بیتی AQQ_{-1} به صورت یکپارچه به اندازه یک بیت به سمت راست شیفت حسابی^۷ داده می‌شود.
 - اگر مقادیر برابر با ۰۱ باشند: ابتدا مقدار M با A جمع می‌شود ($A = A + M$) و سپس ترکیب ۹ بیتی AQQ_{-1} به سمت راست شیفت داده می‌شود.
 - اگر مقادیر برابر با ۱۰ باشند: مقدار M از A تفریق می‌شود ($A = A - M$) و سپس ترکیب ۹ بیتی AQQ_{-1} به سمت راست شیفت داده می‌شود.

۲.۱ مثال روش پایه^۸: ضرب عدد ۷ در ۳

در این مثال ۴ بیتی، فرض می‌کنیم $M = ۰۱۱۱$ (عدد ۷) و $Q = ۰۰۱۱$ (عدد ۳) است. بنابراین مقدار مکمل آن، یعنی $-M = ۱۰۰۱$ خواهد بود. زمان اجرای این روش همواره برابر با تعداد بیت‌ها است؛ یعنی برای اعداد ۴ بیتی دقیقاً ۴ چرخه ساعت نیاز است.

مرحله	$Q_0.Q_{-1}$	Q_{-1}	Q	A	اقدام بعدی
۰	۱۰	۰	۰۰۱۱	۰۰۰۰	وضعیت ۱۰ ← تفریق و سپس شیفت
۱	۱۱	۱	۱۰۰۱	۱۱۰۰	وضعیت ۱۱ ← فقط شیفت
۲	۰۱	۱	۰۱۰۰	۱۱۱۰	وضعیت ۰۱ ← جمع و سپس شیفت
۳	۰۰	۰	۱۰۱۰	۰۰۱۰	وضعیت ۰۰ ← فقط شیفت
۴	-	۰	۰۱۰۱	۰۰۰۱	پایان عملیات

جدول ۱: مراحل ضرب ۷×۳ با استفاده از روش پایه

جواب نهایی از ترکیب ثبات‌های A و Q به دست می‌آید: ۰۰۰۱۰۱۰۱ که در مبنای ده‌دهی برابر با عدد ۲۱ می‌باشد.

¹ Algorithm

² Two's Complement

³ Register

⁴ Multiplicand

⁵ Multiplier

⁶ Accumulator

⁷ Arithmetic Shift Right

⁸ Standard

۲ روش بوٲ بهینه‌شده

در روش پایه، در صورتی که با بلوکی از صفرهای متوالی یا یک‌های متوالی در ضریب مواجه شویم، سیستم تنها عملیات شیفت یک‌بیتی را تکرار می‌کند. در معماری بهینه‌شده این آزمایش، سیستم به جای بررسی مرزهای تک‌بیتی، کل بلوک‌های صفر و یک متوالی در ثبات Q را شناسایی کرده و در یک چرخه ساعت از روی آن‌ها پرش می‌کند. زمان اجرای روش بهینه‌شده به جای مقدار ثابت N چرخه، برابر با تعداد بلوک‌های متوالی در ضریب است که همواره کمتر یا مساوی روش پایه خواهد بود. در این معماری برای تسریع عملیات، به جای آنکه ثبات نتیجه نهایی (result) در هر گام به سمت راست شیفت بخورد (که رویکرد روش پایه است)، این ثبات کاملاً در جای خود ثابت نگه داشته می‌شود. در عوض، برای حفظ ارزش مکانی اعداد در هنگام جمع و تفریق، خود مقدار مضروب (M) پیش از ورود به محاسبات، به تعداد پرش‌های انجام‌شده تا آن لحظه، به سمت چپ شیفت داده می‌شود. همچنین از آنجایی که ثبات نتیجه نهایی هشت بیتی است، مضروب چهار بیتی باید در یک متغیر میانی بسط علامت یابد تا با ثبات نتیجه قابل جمع باشد. برای پیاده‌سازی این منطق و تشخیص اینکه در هر گام چه تعداد کلاک باید پرش شود، از مدارهای ترکیبی بهره گرفته شده است که متغیرهای آن در ادامه معرفی شده‌اند.

۱.۲ متغیرها و منطق پیاده‌سازی

در این معماری، مدار در هر چرخه کلاک به بیت اول ضریب فعلی (بیت دارای کمترین ارزش مکانی) نگاه می‌کند و بر اساس آن تصمیم‌گیری می‌نماید:

- **تشخیص بلوک ۱ و تفریق:** اگر بیت اول برابر با ۱ باشد، به معنای رسیدن به بلوکی از یک‌های متوالی است. مدار عمل تفریق را انجام داده و سپس عملیات پرش را اجرا می‌کند.
- **تشخیص بلوک ۰ و جمع:** اگر بیت اول برابر با ۰ باشد، به معنای رسیدن به بلوکی از صفرهای متوالی است. مدار عمل جمع را انجام داده و سپس عملیات پرش را اجرا می‌کند.
- **مبنای پرش:** مبنای پرش در این روش، شمارش تعداد بیت‌های مشابه و متوالی در سمت راست ضریب است. با محاسبه تفاضل جایگاه اولین بیت یک و اولین بیت صفر، مدار دقیقاً متوجه می‌شود که چند چرخه بی‌حاصل را باید در یک گام پشت سر بگذارد.

برای اجرای دقیق این عملیات بدون از دست رفتن ارزش مکانی^۹ اعداد، از متغیرهای زیر استفاده شده است:

- **shift_Q:** تعداد بیت‌هایی که در چرخه فعلی باید از روی آن‌ها در ضریب پرش شود (همان مبنای پرش).
- **shift_M:** مجموع کل پرش‌های انجام‌شده تا این لحظه. این متغیر ارزش مکانی مضروب را تنظیم می‌کند.
- **op:** نوع عملیات را تعیین می‌کند (صفر برای جمع، یک برای تفریق).
- **valid:** در صورتی که ضریب با بلوک صفر آغاز شود، این سیگنال صفر می‌شود تا از اجرای جمع با صفر جلوگیری کند. این متغیر در واقع نشان‌دهنده این است که آیا در چرخه فعلی، عملیات محاسباتی (جمع یا تفریق) نیز داریم، یا اینکه سیستم فقط باید عمل شیفت را انجام دهد.
- **result:** ثبات دربرگیرنده نتیجه نهایی که برخلاف روش پایه ثابت است و شیفت نمی‌خورد. در عوض، مقدار مضروب پیش از محاسبات به اندازه shift_M به چپ شیفت داده می‌شود.

۲.۲ مثال روش بهینه‌شده: ضرب عدد ۷ در ۳

ورودی‌ها: $M = ۰۱۱۱$ و $Q = ۰۰۱۱$.

^۹Place Value

result	اقدام بعدی	shift-M	shift-Q	op	valid	Q فعلی	ساعت
۰	-	۰	-	-	-	۰۰۱۱	۰
$۰ - (M \ll ۰) = -۷$	بیت اول ۱ است ← تفریق و شیفت Q به اندازه ۲	۰	۲	۱	۱	۰۰۱۱	۱
$-۷ + (M \ll ۲) = ۲۱$	بیت اول ۰ است ← جمع و شیفت Q به اندازه ۲	۲	۲	۰	۱	۰۰	۲
۲۱	اتمام	۴	-	-	-	-	۳

جدول ۲: مراحل ضرب ۷×۳ با روش بهینه‌شده

عملیات به جای چهار چرخه، در دو چرخه به اتمام رسید و متغیر shift_M به مقدار ۴ رسید که نشان‌دهنده پایان کار است.

۳ پیاده‌سازی سخت‌افزاری و بررسی کدها

در این بخش، معماری مدار به دو بخش کلی واحد کنترل^{۱۰} و مسیر داده^{۱۱} تفکیک گردیده است. تمامی اجزا در قالب پیمانه‌های^{۱۲} مجزا نوشته شده‌اند.

۱.۳ پیمانه واحد کنترل

این پیمانه مغز متفکر مدار است که با بررسی ضریب، دستورات لازم را به مسیر داده ارسال می‌کند.

```
1 module ControlUnit(  
2     input clk,  
3     input rst,  
4     input [3:0] Q,  
5     output done,  
6     output op,  
7     output valid,  
8     output [2:0] shift_Q,  
9     output reg [2:0] shift_M  
10 );  
11  
12 wire [2:0] first_one;  
13 wire [2:0] first_zero;  
14  
15 // Priority encoder logic to find first '1'  
16 assign first_one = (Q[0] == 1) ? 3'b000 :  
17                     (Q[1] == 1) ? 3'b001 :  
18                     (Q[2] == 1) ? 3'b010 :  
19                     (Q[3] == 1) ? 3'b011 : 3'b100;  
20  
21 // Priority encoder logic to find first '0'  
22 assign first_zero = (Q[0] == 0) ? 3'b000 :  
23                     (Q[1] == 0) ? 3'b001 :  
24                     (Q[2] == 0) ? 3'b010 :  
25                     (Q[3] == 0) ? 3'b011 : 3'b100;  
26  
27 // Determine operation based on LSB  
28 assign op = (Q[0] == 0) ? 0 : 1;  
29  
30 // Disable addition if starting with a block of zeros  
31 assign valid = ~(shift_M == 0) && (Q[0] == 0);  
32  
33 // Calculate jump size  
34 assign shift_Q = (Q[0] == 0) ? first_one - first_zero : first_zero - first_one;  
35  
36 // Accumulate total shifts synchronously  
37 always @(posedge clk) begin  
38     if (rst) begin  
39         shift_M <= 0;  
40     end else begin  
41         if (!done) begin  
42             shift_M <= shift_M + shift_Q;  
43         end  
44     end  
45 end  
46  
47 // Flag generation when 4 bits are processed  
48 assign done = (shift_M >= 3'b100) ? 1 : 0;  
49
```

¹⁰Control Unit

¹¹Datapath

¹²Module

کد پیمانه واحد کنترل

- خطوط ۲ تا ۴ (ورودی‌ها): سیگنال ساعت (clk)، بازنشانی (rst) و وضعیت ۴ بیتی فعلی ضرب (Q) به عنوان ورودی دریافت می‌شوند.
- خطوط ۵ تا ۹ (خروجی‌ها): سیگنال پایان عملیات (done)، نوع عملگر (op)، مجوز محاسبه (valid)، میزان پرش در ضرب (shift_Q) و متغیر shift_M که مجموع پرش‌های انجام‌شده تا این لحظه را در خود نگه می‌دارد، تعریف شده‌اند.
- خطوط ۱۲ تا ۲۴: در این بخش دو سیم داخلی تعریف شده است. مدارهای منطقی ترکیبی (شبه به رمزگذار اولویت‌دار) از راست به چپ ثبات Q را بررسی می‌کنند تا جایگاه (اندیس) اولین بیت یک (first_one) و اولین بیت صفر (first_zero) را پیدا کنند.
- خط ۲۶: متغیر op با بررسی اولین بیت ضرب تعیین می‌شود، اگر صفر باشد عمل جمع (صفر) و اگر یک باشد عمل تفریق (یک) انتخاب می‌گردد.
- خط ۲۸: سیگنال valid ارزیابی می‌کند که آیا در اولین چرخه (shift_M == 0) قرار داریم و عدد نیز با صفر شروع شده است یا خیر. این سیگنال مشخص می‌کند که آیا در این کلاک به محاسبات حسابی (جمع و تفریق) نیاز داریم، یا اینکه مدار فقط باید عملیات شیفت را انجام دهد.
- خط ۳۰: مقدار پرش (shift_Q) از طریق تفاضل اندیس اولین صفر و اولین یک به دست می‌آید که سبب عبور یک‌باره از روی بلوک‌های تکراری می‌شود.
- خطوط ۳۲ تا ۴۰: این بلوک همگام با ساعت عمل می‌کند. در صورت فعال بودن rst، مقدار shift_M صفر می‌شود و در غیر این صورت، در هر چرخه مقدار پرش جدید (shift_Q) به مقدار مجموع پرش‌های قبلی shift_M افزوده می‌شود تا ارزش مکانی عدد برای محاسبات بعدی حفظ گردد.
- خط ۴۲: سیگنال done بررسی می‌کند که آیا مجموع شیفت‌ها به عدد ۴ (اندازه بیت‌های ورودی) رسیده است یا خیر.

۲.۳ پیمانه مسیر داده

این پیمانه فاقد تصمیم‌گیری منطقی بوده و صرفاً وظیفه اجرای محاسبات ریاضی و عملیات شیفت‌دهی را بر عهده دارد.

```

1 module Datapath(
2     input clk,
3     input rst,
4     input done,
5     input op,
6     input valid,
7     input [2:0] shift_Q,
8     input [2:0] shift_M,
9     input [3:0] M,
10    input [3:0] Q_in,
11    output reg [7:0] result,
12    output reg [3:0] Q_out
13 );
14
15    reg [7:0] M_extended; // Sign-extended multiplicand
16
17    always @(posedge clk) begin
18        if (rst) begin
19            // Reset conditions
20            result <= 8'b0;
21            Q_out <= Q_in;
22            M_extended <= {{4{M[3]}}, M}; // Sign extension to 8 bits

```

```

۲۳     end else begin
۲۴         if (!done) begin
۲۵             // Shift multiplier logic right
۲۶             Q_out <= Q_out >> shift_Q;
۲۷
۲۸             // Arithmetic operation block
۲۹             if (valid) begin
۳۰                 if (op == 0) begin
۳۱                     result <= result + (M_extended << shift_M);
۳۲                 end else begin
۳۳                     result <= result - (M_extended << shift_M);
۳۴                 end
۳۵             end
۳۶         end
۳۷     end
۳۸ end
۳۹ endmodule
۴۰

```

کد پیمانه مسیر داده

- خطوط ۲ تا ۱۰ (ورودی‌ها): علاوه بر ساعت و بازنشانی، این پیمانه مضروب (M) و ضریب اولیه (Q_in) را از کاربر و تمامی فرمان‌های صادرشده (done, op, valid, shift_Q, shift_M) را از واحد کنترل دریافت می‌کند.
- خطوط ۱۱ تا ۱۲ (خروجی‌ها): ثبات ۸ بیتی result برای نگهداری جواب نهایی و ثبات Q_out برای ارسال باقیمانده‌ی ضریب به واحد کنترل در نظر گرفته شده‌اند.
- خط ۱۵: یک ثبات میانی ۸ بیتی به نام M_extended جهت انجام عمل بسط علامت تعریف شده است. از آنجا که این مقدار باید با ثبات نتیجه (result) که هشت بیتی است جمع یا تفریق شود، لازم است M نیز پیش از محاسبه به هشت بیت گسترش یابد تا مدار دچار سرریز^{۱۳} نگردد.
- خطوط ۱۸ تا ۲۱: در صورت فعال شدن rst، جواب نهایی صفر شده، ضریب اولیه در ثبات وضعیت قرار می‌گیرد و از همه مهم‌تر، متغیر ۴ بیتی M با کپی کردن بیت ارزش مکانی علامت، به یک قالب ۸ بیتی (M_extended) گسترش می‌یابد.
- خط ۲۴: در وضعیت کار عادی، ثبات ضریب به اندازه shift_Q به راست شیفت حسابی داده می‌شود تا بیت‌های پردازش‌شده دور ریخته شوند.
- خطوط ۲۶ تا ۳۲: در صورت دریافت مجوز (valid)، با توجه به نوع عملگر (op)، مقدار بسط‌یافته‌ی مضروب ابتدا به میزان shift_M به چپ شیفت داده شده (جهت تنظیم ارزش مکانی) و سپس با ثبات result جمع یا از آن تفریق می‌گردد.

۳.۳ پیمانه سطح بالا

این پیمانه نقش سیم‌کشی و اتصال دو بخش پیشین را بر عهده دارد.

```

۱ module Top(
۲     input clk,
۳     input rst,
۴     input [3:0] M,
۵     input [3:0] Q_in,
۶     output done,
۷     output [7:0] result
۸ );
۹
۱۰ // Internal wires for interconnection
۱۱ wire [2:0] shift_Q;

```

^{۱۳}Overflow


```

۱۲ wire [2:0] shift_M;
۱۳ wire op;
۱۴ wire valid;
۱۵ wire [3:0] Q_wire;
۱۶
۱۷ // Instantiate Control Unit
۱۸ ControlUnit control_unit (
۱۹     .clk(clk),
۲۰     .rst(rst),
۲۱     .Q(Q_wire),
۲۲     .done(done),
۲۳     .op(op),
۲۴     .valid(valid),
۲۵     .shift_Q(shift_Q),
۲۶     .shift_M(shift_M)
۲۷ );
۲۸
۲۹ // Instantiate Datapath
۳۰ Datapath datapath (
۳۱     .clk(clk),
۳۲     .rst(rst),
۳۳     .done(done),
۳۴     .op(op),
۳۵     .valid(valid),
۳۶     .shift_Q(shift_Q),
۳۷     .shift_M(shift_M),
۳۸     .M(M),
۳۹     .Q_in(Q_in),
۴۰     .result(result),
۴۱     .Q_out(Q_wire)
۴۲ );
۴۳
۴۴ endmodule

```

کد پیمانه سطح بالا

- خطوط ۲ تا ۷: درگاه‌های^{۱۴} اصلی ارتباط مدار با کاربر نهایی تعریف شده‌اند که تنها شامل ورودی‌های اصلی، ساعت، بازنشانی و خروجی‌های نهایی (result و done) هستند.
- خطوط ۱۰ تا ۱۴: سیم‌های واسط (wire) برای اتصال درگاه‌های داخلی واحد کنترل و مسیر داده به یکدیگر تعریف شده‌اند تا انتقال فرمان‌ها درون سیستمی باقی بماند.
- خطوط ۱۶ تا ۲۵: پیمانه ControlUnit فراخوانی شده و پورت‌های آن به سیم‌های واسط متصل می‌گردند. ورودی Q در این بخش از طریق سیم Q_wire مستقیماً به خروجی مسیر داده^{۱۵} متصل است.
- خطوط ۲۷ تا ۳۹: پیمانه Datapath فراخوانی شده و درگاه‌های آن تنظیم می‌گردند. خروجی Q_out در این بخش، اطلاعات لحظه‌ای ضریب را بر روی سیم Q_wire قرار می‌دهد تا واحد کنترل در هر لحظه توانایی خواندن آن را داشته باشد.

^{۱۴}Ports

^{۱۵}Datapath

۴ آزمایش و میزآزمون

برای بررسی صحت عملکرد مدار، یک میزآزمون^{۱۶} تدوین گردید. این بستر مقادیر مختلفی شامل اعداد مثبت، منفی و صفر را شبیه‌سازی می‌کند.

```
1 module tb;
2     reg clk;
3     reg rst;
4     reg signed [3:0] M;
5     reg signed [3:0] Q_in;
6     wire done;
7     wire signed [7:0] result;
8
9     // Instantiate Top Module
10    Top uut (
11        .clk(clk),
12        .rst(rst),
13        .M(M),
14        .Q_in(Q_in),
15        .done(done),
16        .result(result)
17    );
18
19    // Clock Generation
20    always #5 clk = ~clk;
21
22    initial begin
23        clk = 0;
24
25        // Test Case 1: M=7, Q_in=5, result=35
26        rst = 1;
27        M = 7;
28        Q_in = 5;
29        #10;
30        rst = 0;
31        wait (done);
32        #20;
33
34        // Test Case 2: M=-4, Q_in=-6, result=24
35        rst = 1;
36        M = -4;
37        Q_in = -6;
38        #10;
39        rst = 0;
40        wait (done);
41        #20;
42
43        // Test Case 3: M=7, Q_in=-3, result=-21
44        rst = 1;
45        M = 7;
46        Q_in = -3;
47        #10;
48        rst = 0;
49        wait (done);
50        #20;
51
52        // Test Case 4: M=-5, Q_in=3, result=-15
53        rst = 1;
54        M = -5;
55        Q_in = 3;
56        #10;
```

¹⁶Testbench

```

۵۷     rst = 0;
۵۸     wait (done);
۵۹     #20;
۶۰
۶۱     // Test Case 5: M=0, Q_in=-7, result=0
۶۲     rst = 1;
۶۳     M = 0;
۶۴     Q_in = -7;
۶۵     #10;
۶۶     rst = 0;
۶۷     wait (done);
۶۸     #20;
۶۹
۷۰     // Test Case 6: M=-8, Q_in=7, result=-56
۷۱     rst = 1;
۷۲     M = -8;
۷۳     Q_in = 7;
۷۴     #10;
۷۵     rst = 0;
۷۶     wait (done);
۷۷     #20;
۷۸
۷۹     // Test Case 7: M=5, Q_in=-1, result=-5
۸۰     rst = 1;
۸۱     M = 5;
۸۲     Q_in = -1;
۸۳     #10;
۸۴     rst = 0;
۸۵     wait (done);
۸۶     #20;
۸۷
۸۸     $stop;
۸۹     end
۹۰     endmodule

```

کد پیمانانه میزآزمون

۱.۴ تحلیل موارد آزمون

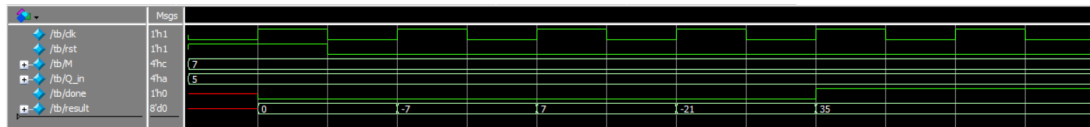
در ادامه هر هفت مورد آزمون^{۱۷} تعریف شده بررسی می‌گردد.

۱.۱.۴ مورد آزمون نخست: ضرب عدد ۷ در ۵

ورودی‌ها: $M = ۰۱۱۱$ (عدد ۷) و $Q = ۰۱۰۱$ (عدد ۵).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
$۰ - (۷ \ll ۰) = -۷$	تفریق	۰	۱	۱	۱	۰۱۰۱	۱
$-۷ + (۷ \ll ۱) = ۷$	جمع	۱	۱	۰	۱	۰۱۰	۲
$۷ - (۷ \ll ۲) = -۲۱$	تفریق	۲	۱	۱	۱	۰۱	۳
$-۲۱ + (۷ \ll ۳) = ۳۵$	جمع	۳	۱	۰	۱	۰	۴
۳۵	اتمام	۴	-	-	-	-	۵

^{۱۷}Test Case

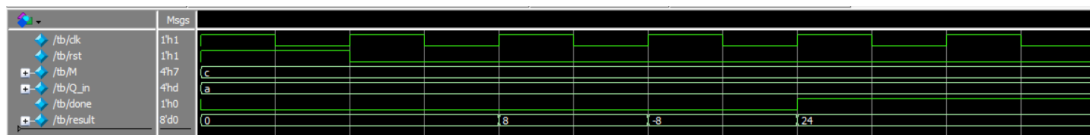


شکل ۱: شکل موج مربوط به مورد آزمون نخست

۲.۱.۴ مورد آزمون دوم: ضرب عدد ۴- در ۶-

ورودی‌ها: $M = ۱۱۰۰$ (عدد ۴-) و $Q = ۱۰۱۰$ (عدد ۶-).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
۰	-	۰	۱	-	۰	۱۰۱۰	۱
$۰ - (-4 \ll 1) = ۸$	تفریق	۱	۱	۱	۱	۱۰۱	۲
$۸ + (-4 \ll 2) = -۸$	جمع	۲	۱	۰	۱	۱۰	۳
$-۸ - (-4 \ll 3) = ۲۴$	تفریق	۳	۱	۱	۱	۱	۴
۲۴	اتمام	۴	-	-	-	-	۵

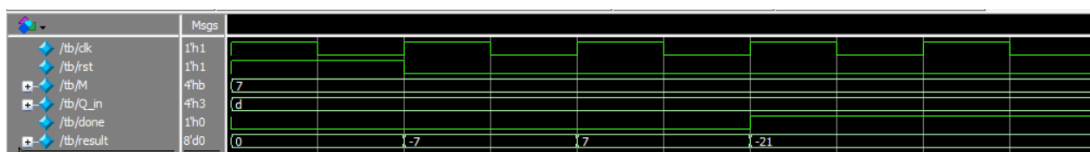


شکل ۲: شکل موج مربوط به مورد آزمون دوم

۳.۱.۴ مورد آزمون سوم: ضرب عدد ۷ در ۳-

ورودی‌ها: $M = ۰۱۱۱$ (عدد ۷) و $Q = ۱۱۰۱$ (عدد ۳-).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
$۰ - (7 \ll 0) = -۷$	تفریق	۰	۱	۱	۱	۱۱۰۱	۱
$-۷ + (7 \ll 1) = ۷$	جمع	۱	۱	۰	۱	۱۱۰	۲
$۷ - (7 \ll 2) = -۲۱$	تفریق	۲	۲	۱	۱	۱۱	۳
-۲۱	اتمام	۴	-	-	-	-	۴

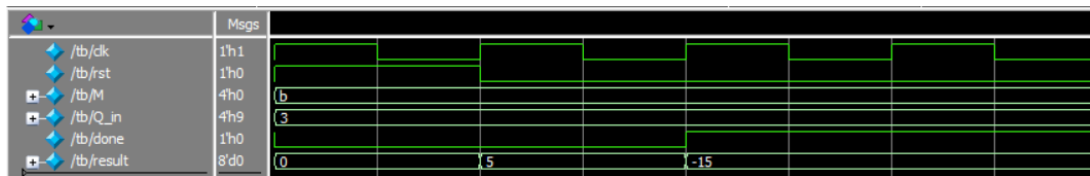


شکل ۳: شکل موج مربوط به مورد آزمون سوم

۴.۱.۴ مورد آزمون چهارم: ضرب عدد ۵- در ۳

ورودی‌ها: $M = ۱۰۱۱$ (عدد ۵-) و $Q = ۰۰۱۱$ (عدد ۳).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
$۰ - (-5 \ll 0) = ۵$	تفریق	۰	۲	۱	۱	۰۰۱۱	۱
$۵ + (-5 \ll 2) = -۱۵$	جمع	۲	۲	۰	۱	۰۰	۲
-۱۵	اتمام	۴	-	-	-	-	۳

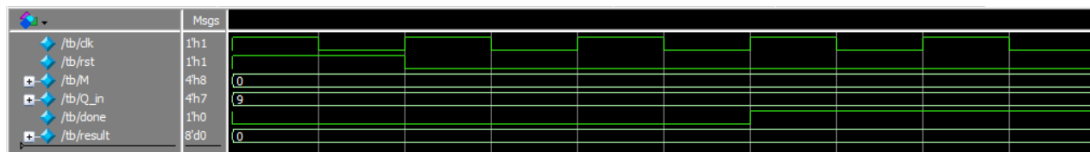


شکل ۴: شکل موج مربوط به مورد آزمون چهارم

۵.۱.۴ مورد آزمون پنجم: ضرب عدد ۰ در ۷-

ورودی‌ها: $M = ۰۰۰۰$ (عدد ۰) و $Q = ۱۰۰۱$ (عدد ۷-).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
$۰ - (۰) = ۰$	تفریق	۰	۱	۱	۱	۱۰۰۱	۱
$۰ + (۰) = ۰$	جمع	۱	۲	۰	۱	۱۰۰	۲
$۰ - (۰) = ۰$	تفریق	۳	۱	۱	۱	۱	۳
۰	اتمام	۴	-	-	-	-	۴

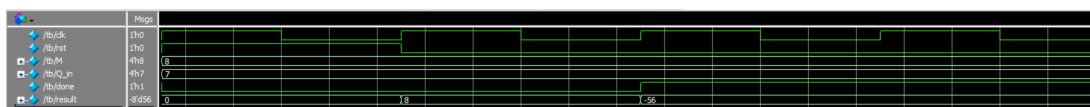


شکل ۵: شکل موج مربوط به مورد آزمون پنجم

۶.۱.۴ مورد آزمون ششم: ضرب عدد ۸- در ۷

ورودی‌ها: $M = ۱۰۰۰$ (عدد ۸-) و $Q = ۰۱۱۱$ (عدد ۷).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
$۰ - (-۸ \ll ۰) = ۸$	تفریق	۰	۳	۱	۱	۰۱۱۱	۱
$۸ + (-۸ \ll ۳) = -۵۶$	جمع	۳	۱	۰	۱	۰	۲
۵۶-	اتمام	۴	-	-	-	-	۳

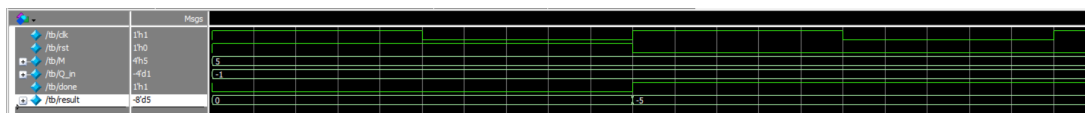


شکل ۶: شکل موج مربوط به مورد آزمون ششم

۷.۱.۴ مورد آزمون هفتم: ضرب عدد ۵ در ۱-

ورودی‌ها: $M = ۰۱۰۱$ (عدد ۵) و $Q = ۱۱۱۱$ (عدد ۱-).

result	عملیات	shift_M	shift_Q	op	valid	Q فعلی	ساعت
$۰ - (۵ \ll ۰) = -۵$	تفریق	۰	۴	۱	۱	۱۱۱۱	۱
۵-	اتمام	۴	-	-	-	-	۲



شکل ۷: شکل موج مربوط به مورد آزمون هفتم

۵ بررسی قابلیت سنتز در نرم افزار Quartus

یک پروژه در نرم افزار Quartus ساخته شد و فایل های طراحی به آن افزوده شد. نام پیمانه سطح بالا^{۱۸} انتخاب شد. برای بررسی اولیه قابل سنتز بودن طراحی، مرحله تحلیل و سنتز از مسیر:

Processing > Start > Start Analysis & Synthesis

اجرا شد. سپس برای تولید نتایج مرحله تطبیق و امکان مشاهده نمای پس از جایابی، کامپایل کامل از مسیر:

Processing > Start Compilation

انجام گرفت.

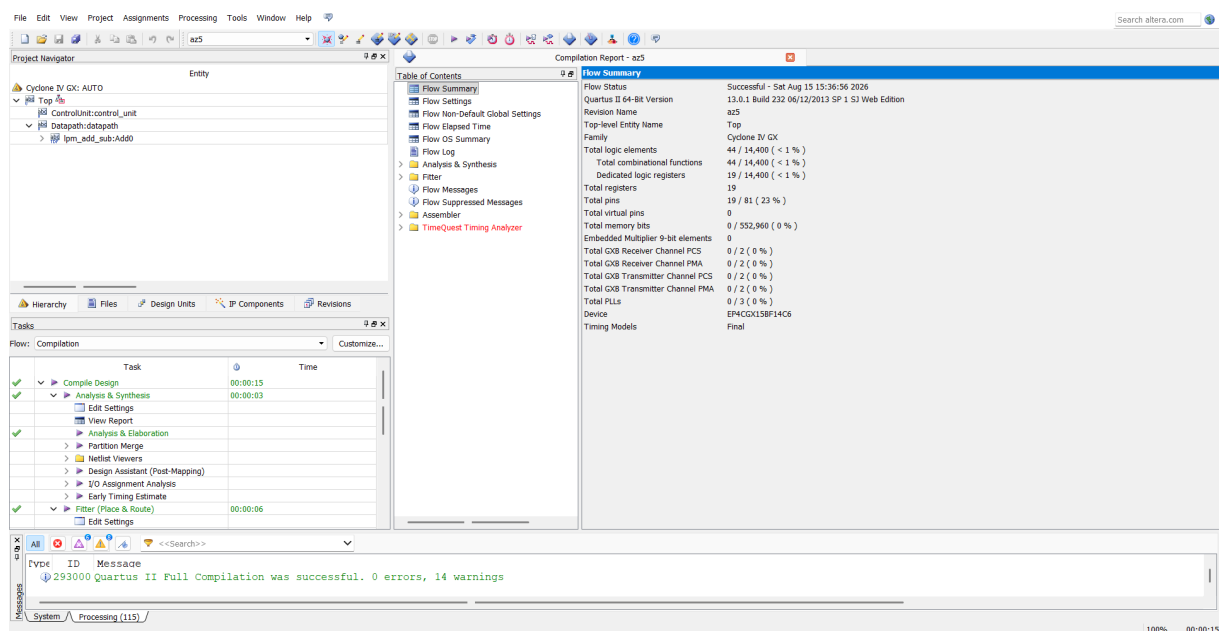
پس از تکمیل کامپایل، سه نمای اصلی از مسیرهای زیر باز شدند:

Tools > Netlist Viewers > RTL Viewer

Tools > Netlist Viewers > Technology Map Viewer (Post-Mapping)

Tools > Netlist Viewers > Technology Map Viewer (Post-Fitting)

در ادامه، خروجی موفقیت آمیز بودن کامپایل و همچنین نماهای بررسی شده به ترتیب آورده شده است. نمای سطح انتقال ثبات در واقع یک فهرست اتصالات^{۱۹} بصری است و نرم افزار Quartus با تشکیل سلسله مراتب^{۲۰} مدار، پروسه را بدون خطا طی کرده است.

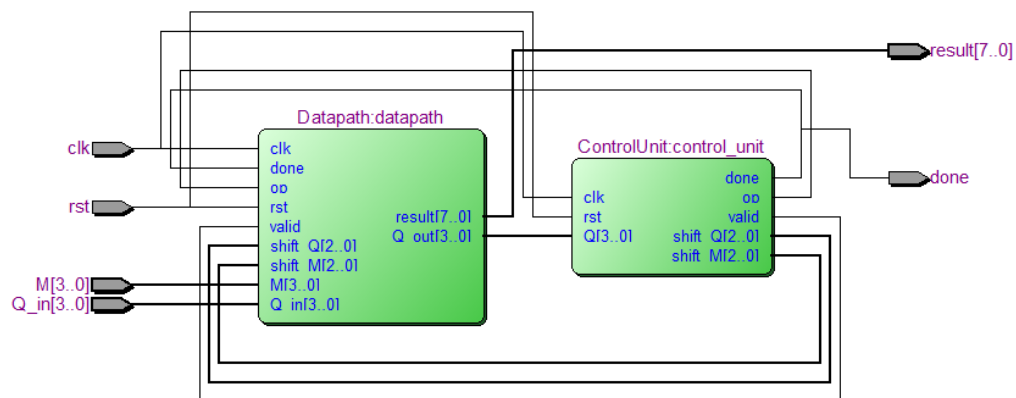


شکل ۸: گزارش موفقیت آمیز بودن کامپایل کامل و فرآیند سنتز در Quartus

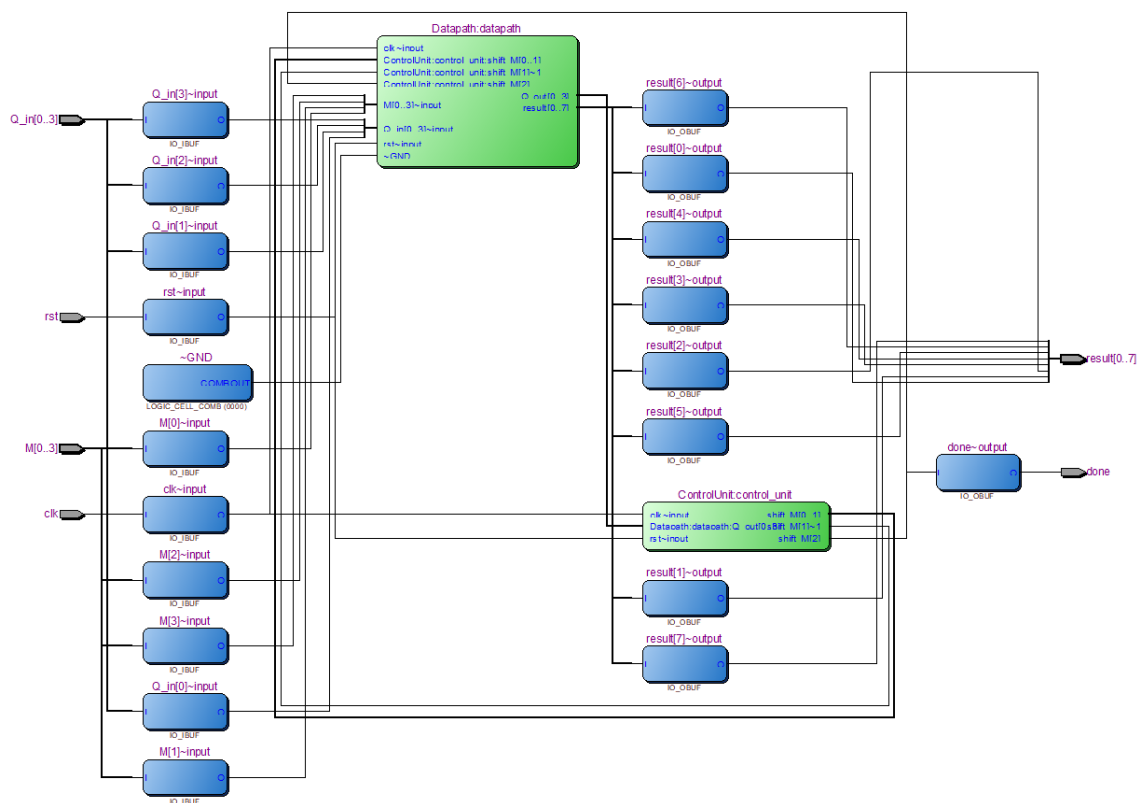
¹⁸Top-Level Entity

¹⁹Netlist

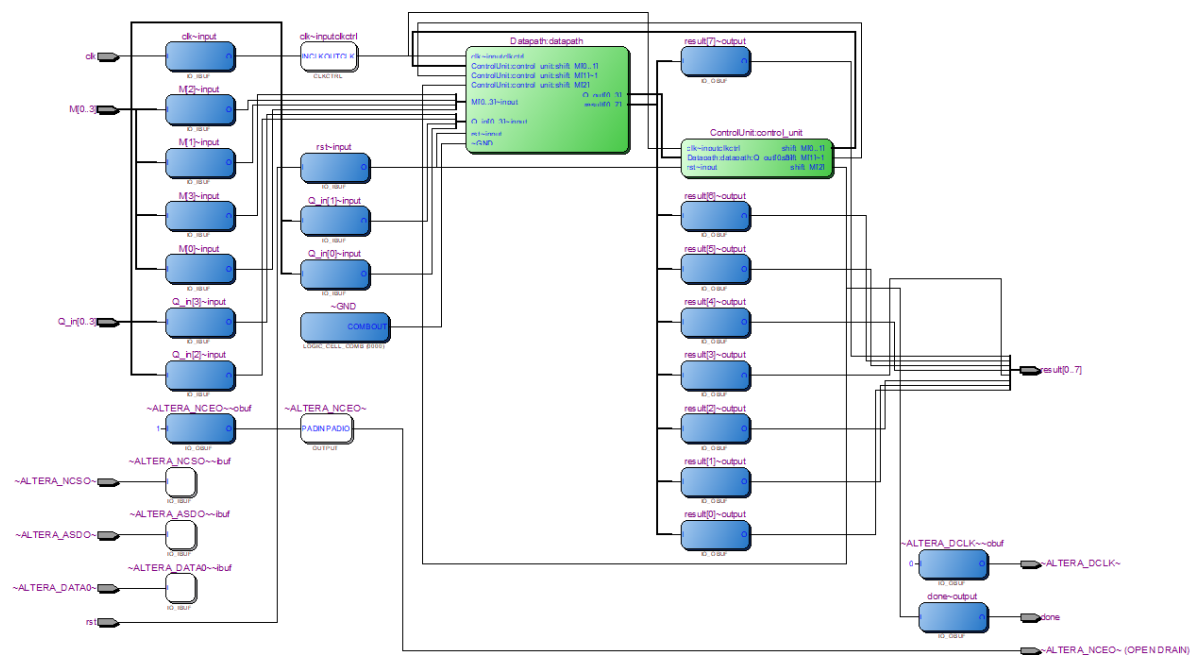
²⁰Hierarchy



شکل ۹: نمای سطح انتقال ثبات (RTL Viewer)



شکل ۱۰: نمای نگاشت فناوری پس از نگاشت (Post-Mapping)



شکل ۱۱: نمای نگاشت فناوری پس از جایی و تطبیق (Post-Fitting)